

Avro & Schema Evolution

A shared event contract • Generated Java types • Safe change boundaries

SAY IT IN THE INTERVIEW

"I use Avro for OrderCreated events and Schema Registry to manage their schemas. I added a source field with the default UNKNOWN, while new events set WEB. That lets a new reader resolve older records that do not contain source. Inventory result events currently remain JSON."

01 WHAT EACH PIECE DOES

- **Avro:** schema-based binary serialization. **Schema Registry:** stores schema versions and supports compatibility checks; Kafka still stores the event records.
- The Confluent Avro serializer writes a schema identifier with encoded data, not the full schema in every record. The consumer resolves the writer schema to decode it.
- Both services have semantically matching **order-created.avsc** files. avro-maven-plugin generates Java classes during **generate-sources**. Do not edit generated classes.

02 OUR ACTUAL EVENT CONTRACT

- Identity: **eventId**, **orderId**. Business data: **productId**, **quantity**, **amount**, **status**. Metadata: **occurredAt**, **source**.
- amount uses decimal with precision **19**, scale **2**; occurredAt uses timestamp-millis. These logical types preserve business meaning beyond raw bytes/longs.
- **orders.created.avro** uses Avro. **inventory.reserved** and **inventory.reservation.failed** use JSON. Do not describe every topic as Avro.

03 EXPLAIN THE CHANGE WITH ONE EXAMPLE

- Old writer: no source field. New reader: source string with default **UNKNOWN**. While reading that older record, the reader supplies UNKNOWN.
- New publisher explicitly sets **WEB**. A reader default supports schema resolution; it does not retroactively rewrite old Kafka records.
- **Backward compatibility:** a new reader can read old data. Forward is the reverse; full requires both directions. Actual Registry subject compatibility must be checked, not assumed from the file alone.

04 FOLLOW-UP QUESTIONS I SHOULD HANDLE

- **Why Avro over plain JSON?** An explicit contract, generated types and controlled evolution. Trade-off: schema tooling and Registry availability become operational dependencies.
- **Registry is down?** A needed registration/schema lookup can fail or block; cached schemas may allow some operations. It is not correct to say all sends always fail immediately.
- **Safe change?** Regenerate/build both services, check compatibility and test old/new records. Defaults do not make every change safe.

YAAD RAKHO: old record + new reader → source UNKNOWN; new event → WEB.

SOURCE CHECK: Both order-created.avsc files + Maven plugins; Avro/Confluent specifications